

Water Conservation CRM — Salesforce & visual studio Implementation

Table of Contents

- Executive Summary
- Project Overview and Goals
- Delivery Methodology and Environment Strategy
- Phase 1: Problem Understanding & Industry Analysis
- Phase 2: Org Setup & Configuration
- Phase 3: Data Modeling & Relationships
- Phase 4: Process Automation (Admin)
- Phase 5: Apex Programming (Developer)
- Phase 6: User Interface Development
- Phase 7: Integration & External Access
- Phase 8: Data Management & Deployment
- Phase 9: Reporting, Dashboards & Security Review
- Phase 10: Quality assurance testing
- Governance, Security & Compliance
- KPIs and Success Metrics
- Appendices (Templates & Checklists)

Executive Summary

This playbook provides an end-to-end guide for planning, implementing, and launching the Water Conservation CRM on Salesforce. It is structured into 10 phases spanning discovery, architecture, configuration, custom development, integration, data, reporting, security, and final presentation. Each phase includes objectives, deliverables, activities, roles, acceptance criteria, risks, and checklists. The guidance follows Salesforce best practices, emphasizes security and data quality, and supports iterative delivery with clear milestones.

Project Overview and Goals

Objectives

- Centralize water conservation program data (customers, site assessments, rebates, conservation measures, field operations).
- Streamline workflows (intake, eligibility, inspections, approvals, payments).
- Improve reporting on conservation impact (water saved, program performance, compliance).
- Enable secure collaboration among departments and external partners.
- Ensure data privacy, auditability, and regulatory compliance.

Scope

- In scope: Salesforce setup, core data model, flows, Apex services, LWCs, API integrations, data migration, reports/dashboards, security hardening, CI/CD.
- Out of scope (initial release): Complex mobile offline, managed packaging for AppExchange, advanced AI features.

Constraints & Assumptions

- Phased rollout by program area; 2–3 week sprints; sandbox-first changes; Git-based version control; CI/CD with environment gates.
- Data from legacy systems available via export/API; SME availability for UAT; security policies approved by InfoSec.

Success Metrics

- Adoption: $\geq 80\%$ weekly active users within 60 days of launch.
- Efficiency: $\geq 30\%$ reduction in average case-to-approval time.
- Data quality: $< 1\%$ critical duplicates; $\geq 95\%$ required fields completed.
- Impact tracking: 100% of approved measures have verified water savings calculations.

Delivery Methodology and Environment Strategy

Approach

- Agile with iterative releases; Definition of Ready/Done at story and phase levels.
- Peer review for config and code; automated tests; quality gates.
- Documentation-first for data model and integration contracts.

Environments & Branching

- Dev sandboxes: Feature development and spikes.
- QA/UAT sandbox: Integrated testing and user acceptance.
- Staging (optional): Release candidate validation, performance tests.
- Production: Controlled releases with back-out plan.
- GitFlow (recommended): main (prod), develop (integration), feature/, *release*/, hotfix/*.

Release Cadence & CI/CD

- CI: Validate, lint, unit tests on pull request.
- CD: Promote metadata and source using SFDX; seed data via scripts; smoke tests post-deploy.
- Change approvals via pull requests + release notes.

Phase 1: Problem Understanding & Industry Analysis

Objective

Establish domain understanding, business objectives, regulatory context, and measurable outcomes to drive solution scope and priorities.

Key Deliverables

- Stakeholder map and RACI
- Business capability map (Programs, Assessments, Rebates, Field Ops, Finance)
- Current-state vs. target-state process maps
- Regulatory & compliance inventory (data retention, PII, audit)
- Success metrics and OKRs
- Prioritized backlog with epics and MVP definition

Activities

- Conduct discovery workshops and SME interviews.
- Analyze industry standards and peer utility programs.
- Map value streams: intake → assessment → approval → realization → reporting.
- Identify system landscape and integration candidates.
- Draft product vision, personas, and journey maps.

Roles

- Product Owner, Business Analyst, Solution Architect, Compliance Lead, Delivery Lead.

Acceptance & Exit Criteria

- Signed off problem statement, scope, and MVP.
- Backlog groomed for Phase 2 kickoff (Definition of Ready met).
- Risks logged with mitigation strategies.

Risks & Mitigations

- Ambiguous scope → timebox workshops; decision log; change control.
- Compliance constraints → early InfoSec review; privacy by design.

Checklist

- Stakeholders identified and engaged
- KPIs defined
- Process maps baselined
- Risk register opened

Phase 2: Org Setup & Configuration

Objective

Establish a secure, governed Salesforce foundation to support subsequent build activities.

Key Deliverables

- Org strategy (production + sandboxes) and access model
- Profiles/Permission Sets and Permission Set Groups
- Named Credentials/External Credentials baseline
- SSO/MFA plan; Session settings; Password policies
- Dev tooling: SFDX, Prettier, ESLint, PMD rules
- Scratch org definition files (if using Source-Tracked DX)

Activities

- Configure company settings, business hours, holidays, fiscal year.
- Set up users, roles, role hierarchy, public groups.
- Establish permission model: least privilege, PS > profiles where possible.
- Enable MFA and SSO (as applicable).
- Create baseline sandboxes; connect CI/CD.
- Define naming conventions and repo structure (force-app/main/default, unlocked packages if used).

Roles

- Admin, Security Architect, DevOps Engineer.

Acceptance & Exit Criteria

- Security baseline validated (Health Check score target, MFA enforced).
- CI pipeline passes for baseline metadata push.
- Access model documented and approved.

Risks & Mitigations

- Over-permissive access → permission reviews; automated policy checks.
- Misaligned environments → env matrix and refresh cadence.

Checklist

- MFA/SSO configured
- Profiles minimized; Permission Sets used
- CI/CD pipeline green
- Audit settings enabled

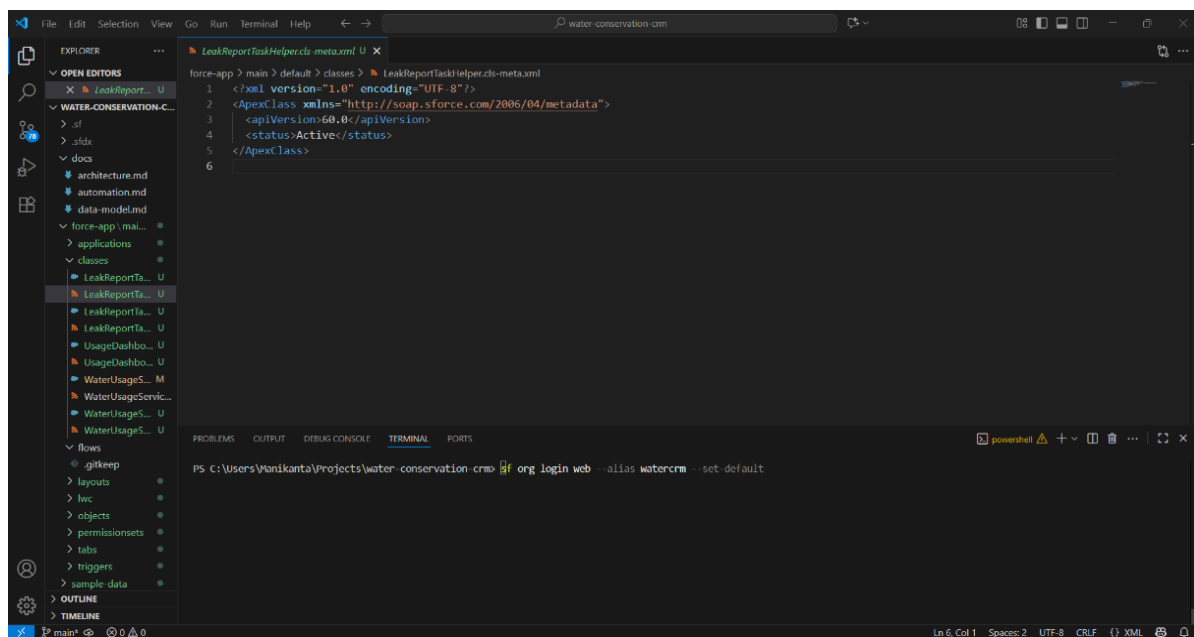


Fig 1: ORG SETUP

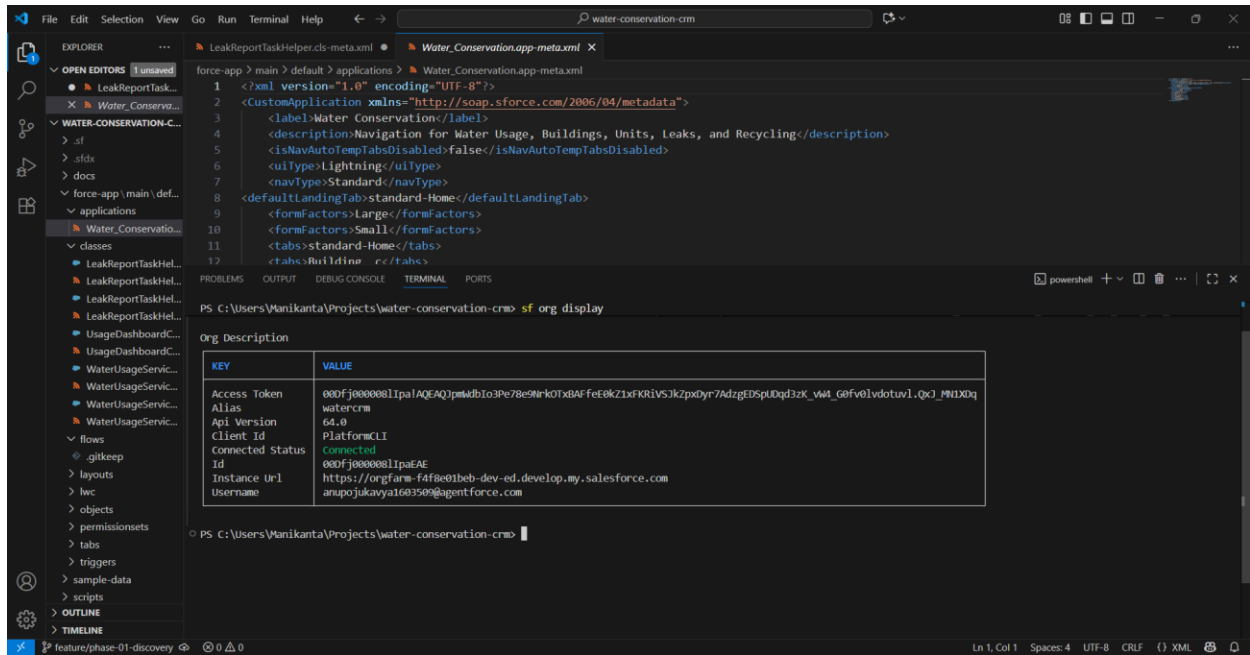


Fig 2: ORG DISPLAY

Phase 3: Data Modeling & Relationships

Objective

Design a scalable, secure data model to capture programs, participants, sites, measures, inspections, approvals, and financials.

Key Deliverables

- ERD with entities and relationships
- Object inventory (standard vs. custom) with rationale
- Field dictionary (API names, labels, types, help text)
- Record Types and Business Processes
- Validation rules and required fields
- Sharing model: OWD, criteria-based shares, teams

Activities

- Leverage standard objects (Accounts, Contacts, Cases) where possible.
- Create custom objects (e.g., Building__c, LeakReport__c, RecyclingLog__c, Unit__c, WaterUsage__c).
- Define lookups/master-detail and junction objects.
- Implement picklists with global value sets; help text for usability.
- Establish data quality rules and duplicate rules/matching rules.

Roles

- Solution Architect, Admin, Data Architect.

Acceptance & Exit Criteria

- ERD and dictionary peer-reviewed and approved.
- Sharing model yields correct record visibility in test scenarios.
- Validation rules cover critical data quality constraints.

Risks & Mitigations

- Over-customization → standard-first review gate.
- Future scalability → big object strategy if needed; selective indexes.

Checklist

- ERD merged to repo
- Object/field naming conventions applied
- Sharing and ownership tested
- Duplicate rules validated

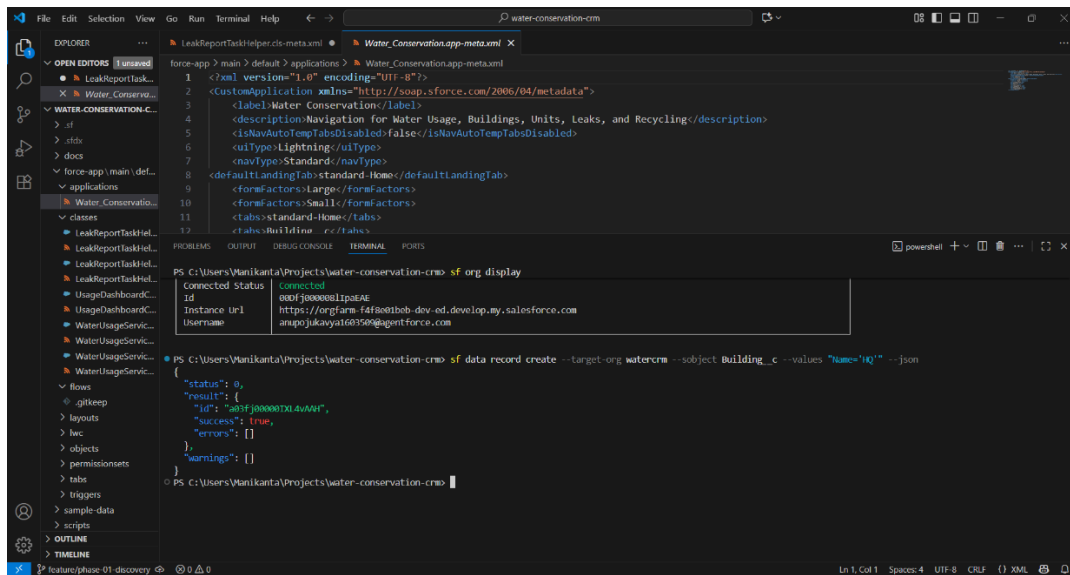


Fig 3: DATA MODELLING

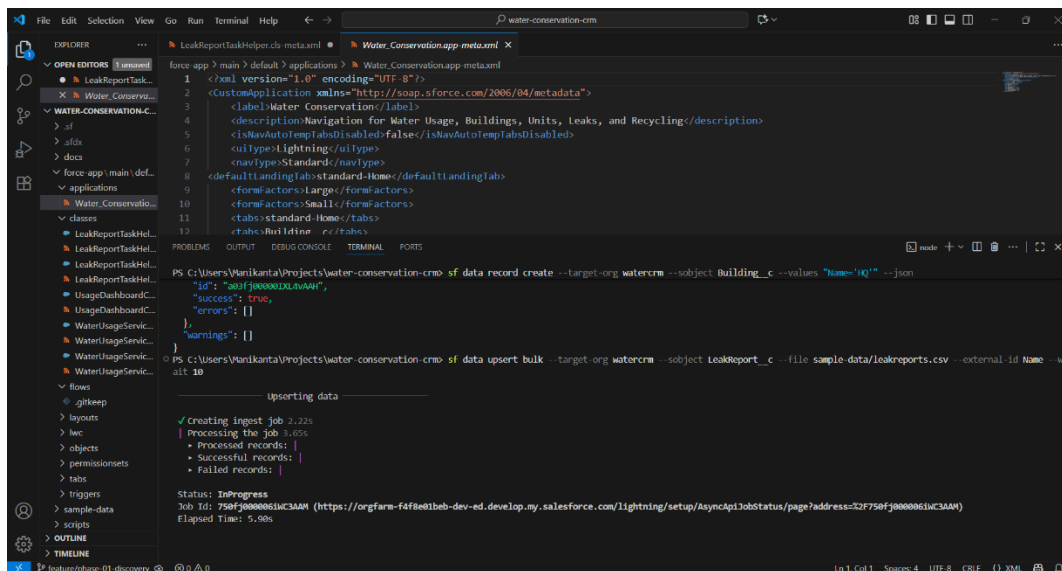


Fig 4: RECORDS PROCESSD

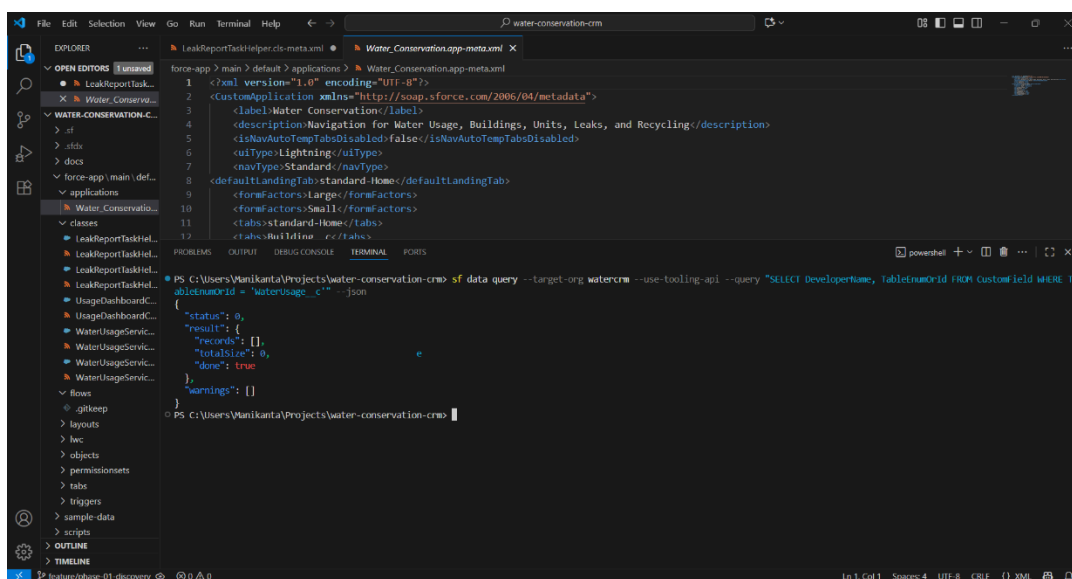


Fig 1: ORG SETUP

The screenshot shows the Salesforce Object Manager interface for the 'Water Usage' object. The 'Fields & Relationships' section is active, displaying a list of 9 fields. The left sidebar contains navigation links for various object settings. The top navigation bar includes 'Setup', 'Home', and 'Object Manager'.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Facility	Facility__c	Lookup(Facility)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Period	Period__c	Picklist		
Quantity	Quantity__c	Number(18, 0)		
Usage Date	Usage_Date__c	Date		
Volume (Liters)	Volume_Liters__c	Number(16, 2)		
Water Usage Record	Name	Auto Number		✓

Fig 5: STATUS OF THE RECORDS

Phase 4: Process Automation (Admin)

Objective

Automate business processes using low-code first (Flows), reserving code for complex or bulk-safe needs.

Key Deliverables

- Record-Triggered Flows (create/update/delete)
- Screen Flows for guided intake and inspections
- Scheduled Flows for nightly syncs and recalculations
- Flow naming/versioning conventions and guardrails
- Declarative rollback approach (where feasible)

Activities

- Implement RT Flows for eligibility checks, status transitions, and derived fields.
- Build Screen Flows for guided applications, measure selection, and inspection forms.
- Use subflows for reuse; avoid recursion; set entry criteria precisely.
- Ensure bulkification in Flows: use collection operations, avoid per-record DML.
- Replace Workflow Rules/Process Builder; centralize in Flow.

Roles

- Admin, Solution Architect, QA.

Acceptance & Exit Criteria

- Flows pass unit and UAT cases; fault paths handled; errors surfaced to users.
- No SOQL/DML limits hit in bulk tests; scheduled flows complete within window.
- Audit logs for critical automations.

Risks & Mitigations

- Flow sprawl → catalog and governance; design reviews.
- Fault handling gaps → standard error handler subflow.

Checklist

- Naming and foldering standards
- Subflows for reusable logic
- Limits testing with data volumes
- Error handling and toasts/messages

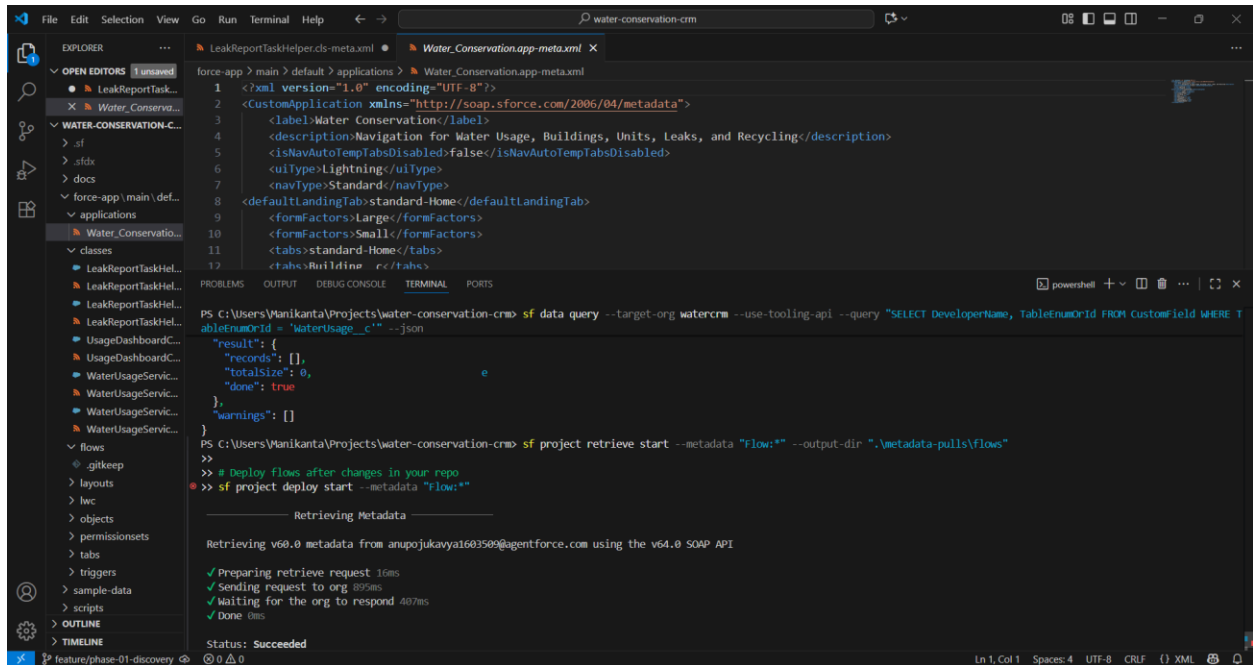


Fig 6: FLOWS

Phase 5: Apex Programming (Developer)

Objective

Implement robust, testable Apex for complex logic, integrations, and performance-sensitive operations.

Key Deliverables

- Service layer classes, domain triggers/handlers
- Queueables/Batch/Schedulable jobs for async/bulk
- Platform Event handling (if required)
- Comprehensive unit tests ($\geq 85\%$ overall; 100% for critical logic)
- Code quality reports (PMD, ESLint for LWC/JS)

Activities

- Adopt Trigger Framework (one trigger per object $\rightarrow$ handler pattern).
- Establish layers: Domain (SObject logic), Service (business ops), Selector (SOQL).
- Use Custom Metadata/Settings for feature flags and thresholds.
- Bulkify and cache reference data; governor-safe patterns.
- Implement error handling and logging (Platform Events, Custom Object, or Error Logging framework).
- Unit tests with FFLIB-like patterns: factory data, asserts on behavior, isolation.

Roles

- Developer, Technical Lead, Code Reviewer, QA.

Acceptance & Exit Criteria

- Tests green with coverage thresholds; static analysis passes.
- No critical PMD issues; peer review completed.
- Async patterns used for long-running tasks; retries where appropriate.

Risks & Mitigations

- Governor limits → bulk patterns; selective fields; indexed filters.
- Flaky tests → deterministic test data, SeeAllData=false, factories.

Checklist

- Trigger framework in place
- Service/selector patterns adopted
- Tests $\geq 85\%$ coverage overall
- PMD and CI checks pass

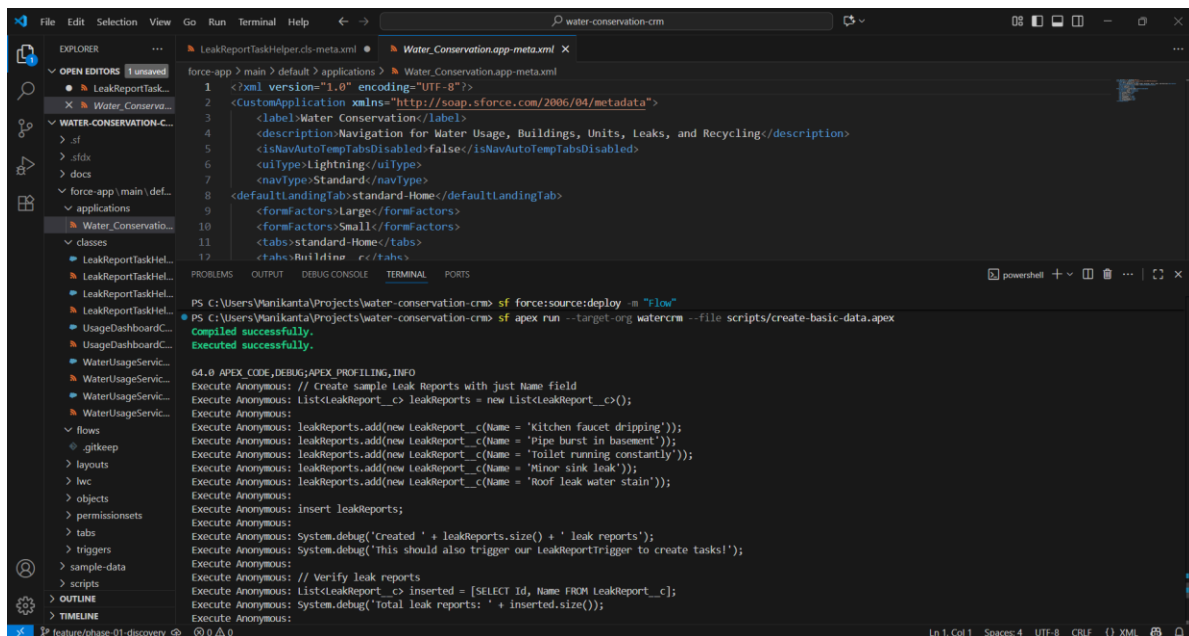


Fig 7: APEX_CODE & DEBUG

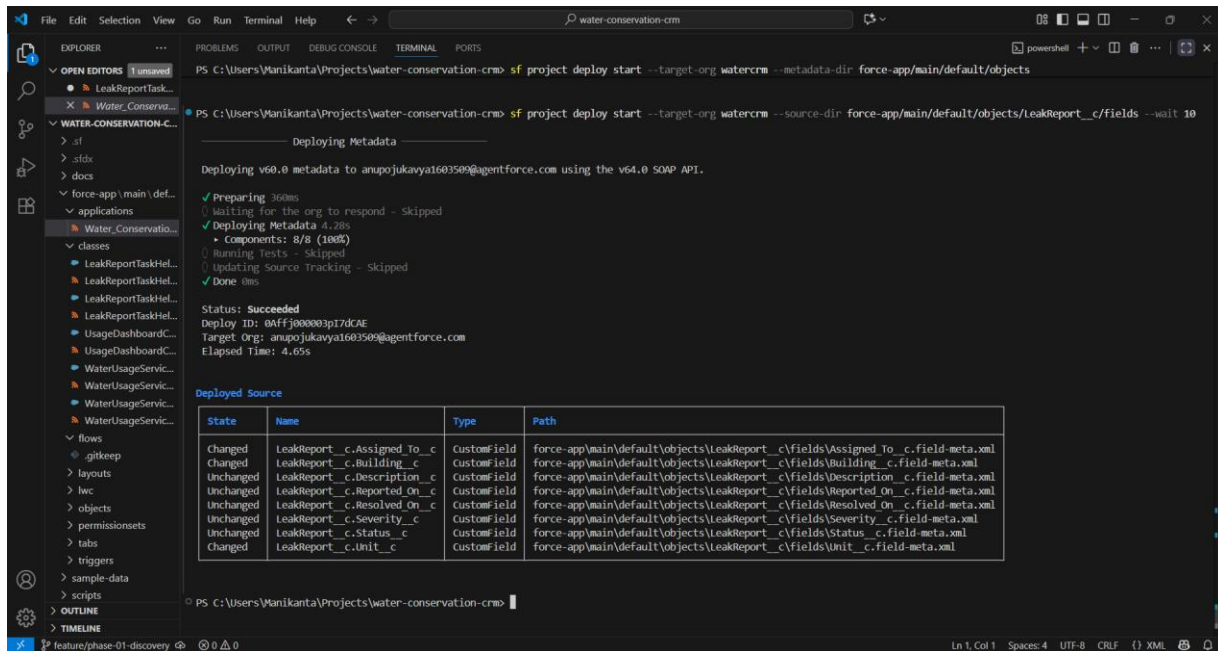


Fig 8: DEPLOYMENT

Phase 6: User Interface Development

Objective

Deliver intuitive, performant UIs using Lightning App Builder, LWCs, and dynamic forms.

Key Deliverables

- App pages, record pages, and utility items
- LWCs for specialized interactions (measure calculators, inspection capture)
- Dynamic forms and visibility rules
- Design system adherence (SLDS); accessibility compliance (WCAG)

Activities

- Layout optimization by persona; quick actions and related lists curated.
- LWC development with SLDS, wire adapters, Lightning Data Service.
- Caching strategies (lightning/ui* APIs; @wire with cache).
- Form validation and helpful error states; toast patterns.
- Performance checks: Lightning Usage App; LWC benchmarks.

Roles

- UX Designer, LWC Developer, Admin, Accessibility Reviewer.

Acceptance & Exit Criteria

- UAT sign-off by personas; Lighthouse/usage metrics within thresholds.
- Mobile responsiveness tested if applicable.
- Accessibility checks pass (keyboard nav, contrast, ARIA).

Risks & Mitigations

- UI sprawl → page templates and governance; component library.
- Performance → lazy loading, LDS, SOQL selectivity.

Checklist

- Persona-based pages
- SLDS-compliant components
- Accessibility verified
- Performance baseline recorded

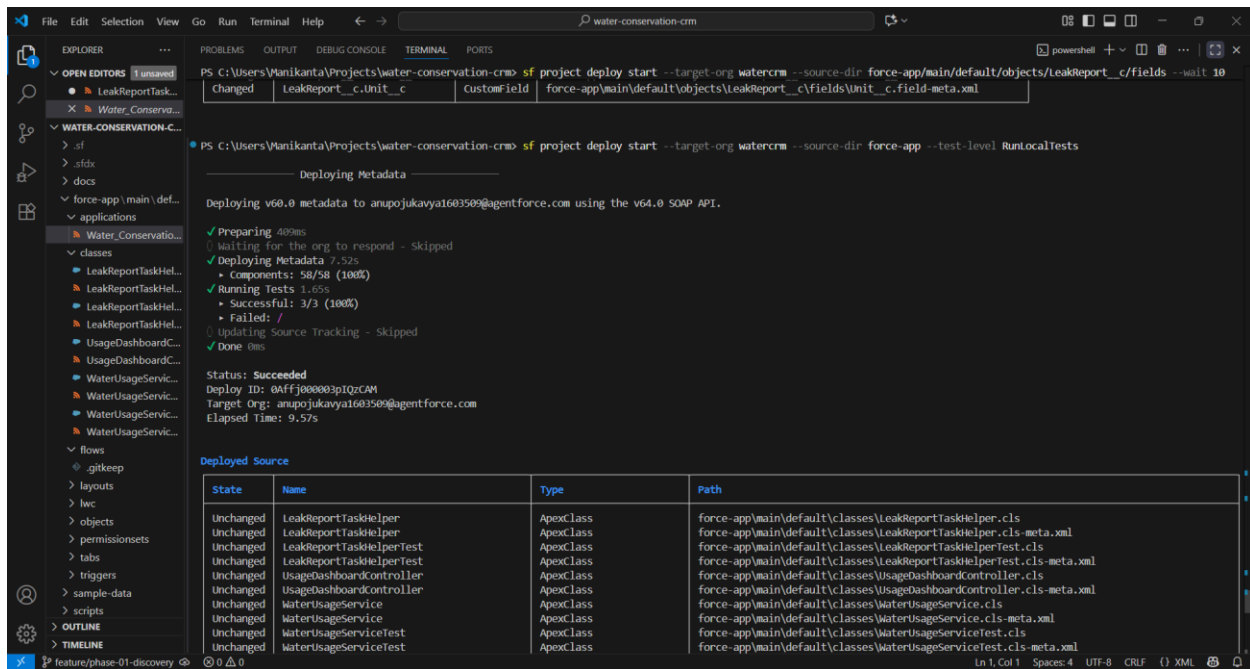


Fig 9: USER INTERFACE

Phase 7: Integration & External Access

Objective

Connect Salesforce with external systems securely and reliably (billing, GIS, document management, payment gateways, analytics).

Key Deliverables

- Integration architecture (patterns, auth, rate limits)
- Named Credentials/External Credentials and Auth Providers
- REST/SOAP services (Apex) and/or External Services (OpenAPI)
- Event-driven patterns (Platform Events/Change Data Capture) where applicable
- Monitoring and alerting for integrations

Activities

- Select patterns: request/response, fire-and-forget, batch sync, event-driven.
- Define contracts (OpenAPI/JSON schemas); version endpoints.
- Implement retry/backoff; idempotency keys; dead-letter handling.
- Use Shield Platform Encryption/Field Encryption as needed.
- Secure external access via Experience Cloud or API Gateways with proper scopes.

Roles

- Integration Architect, Developer, Security Engineer, External Partner.

Acceptance & Exit Criteria

- Pen-tested endpoints; secrets managed (no hardcoding).
- Observability in place (logs, traces, dashboards).
- Error budgets and SLAs defined; graceful degradation paths.

Risks & Mitigations

- Rate limits → queuing/backpressure; composite APIs.
- Secrets exposure → Named Credentials, externalized configs, vaults.

Checklist

- Contracts versioned and documented
- Auth via Named Credentials
- Retry/idempotency implemented
- Monitoring and alerts configured

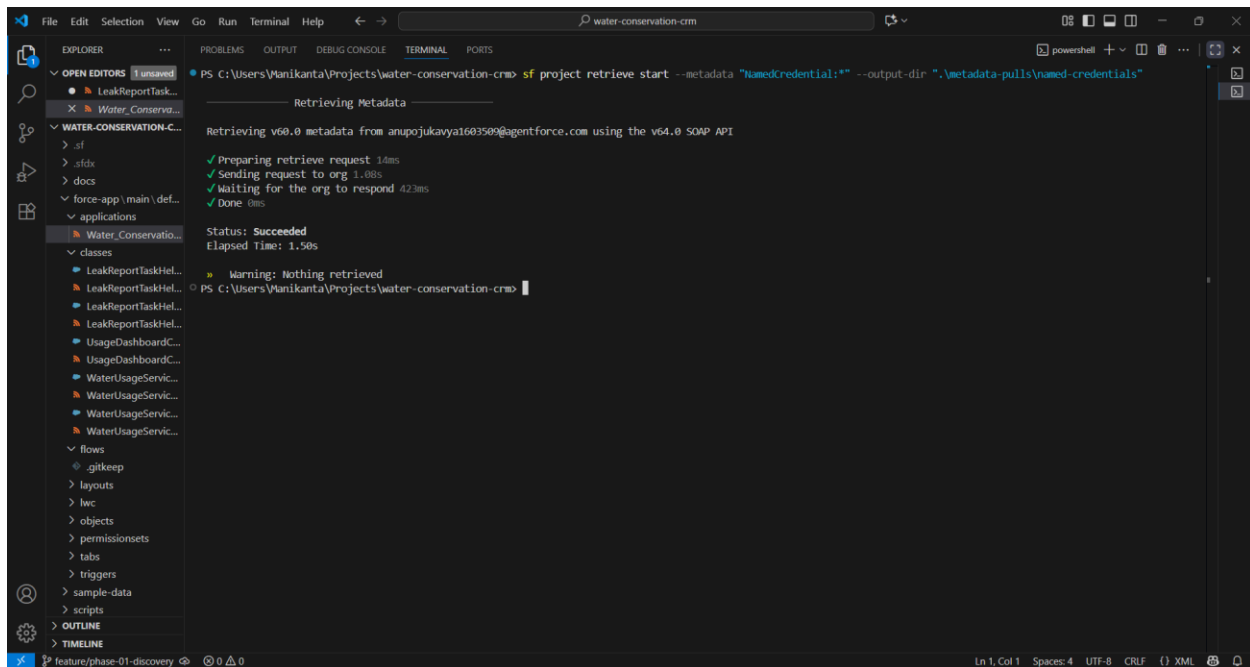


Fig 10: INTEGRATION

Phase 8: Data Management & Deployment

Objective

Migrate, seed, and protect data across environments; deploy changes safely with rollbacks.

Key Deliverables

- Data migration plan (profiles, volumes, sequencing, mapping)
- Data quality rules, deduping strategy
- Deployment runbooks and back-out plan
- Environment data seeding scripts (SFDX data tree, ETL)
- Backup and recovery strategy

Activities

- Define mapping from legacy to Salesforce; transform and cleanse.
- Load in order: reference → parents → children; handle external IDs.
- Use Bulk API 2.0 for large loads; monitor errors; reconcile.
- Practice cutover in UAT/Staging; measure timings.
- Validate profiles/perm sets post-deploy; smoke tests.

Roles

- Data Lead, Admin, DevOps, QA, Business Data Owners.

Acceptance & Exit Criteria

- UAT data migration sign-off; reconciliation within thresholds.
- Deployment rehearsals complete; MTTR and rollback validated.
- Backup/restore tested for critical objects.

Risks & Mitigations

- Dirty legacy data → cleansing rules, SME validation.
- Deployment drift → source of truth in Git, manifest-controlled deploys.

Checklist

- Field mapping and external IDs confirmed
- Dry runs completed
- Reconciliation reports
- Back-out plan tested

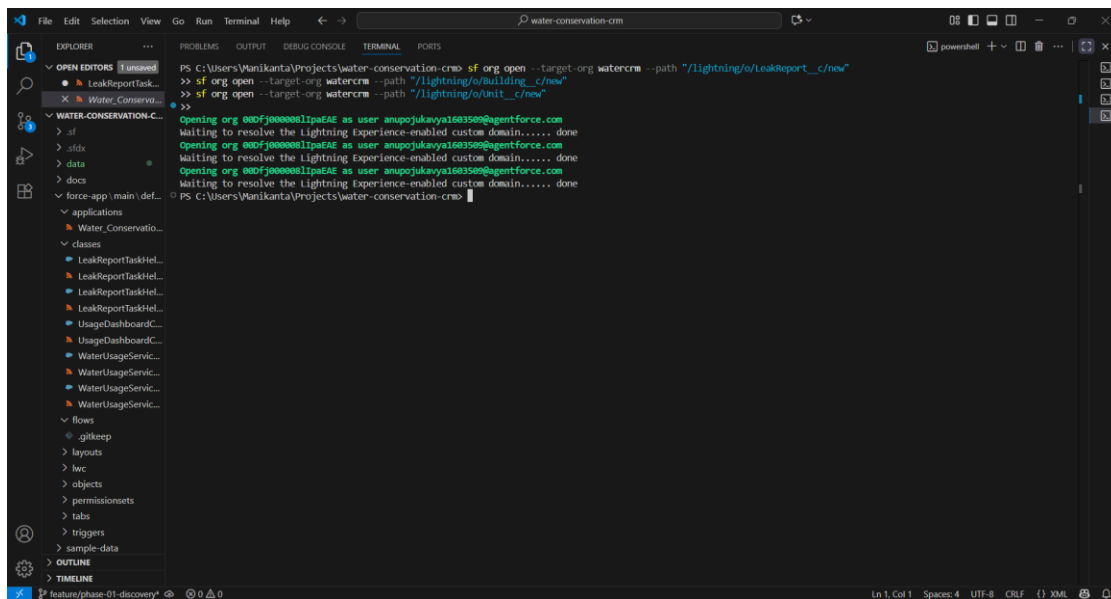


Fig 11: DATA MANAGEMENT

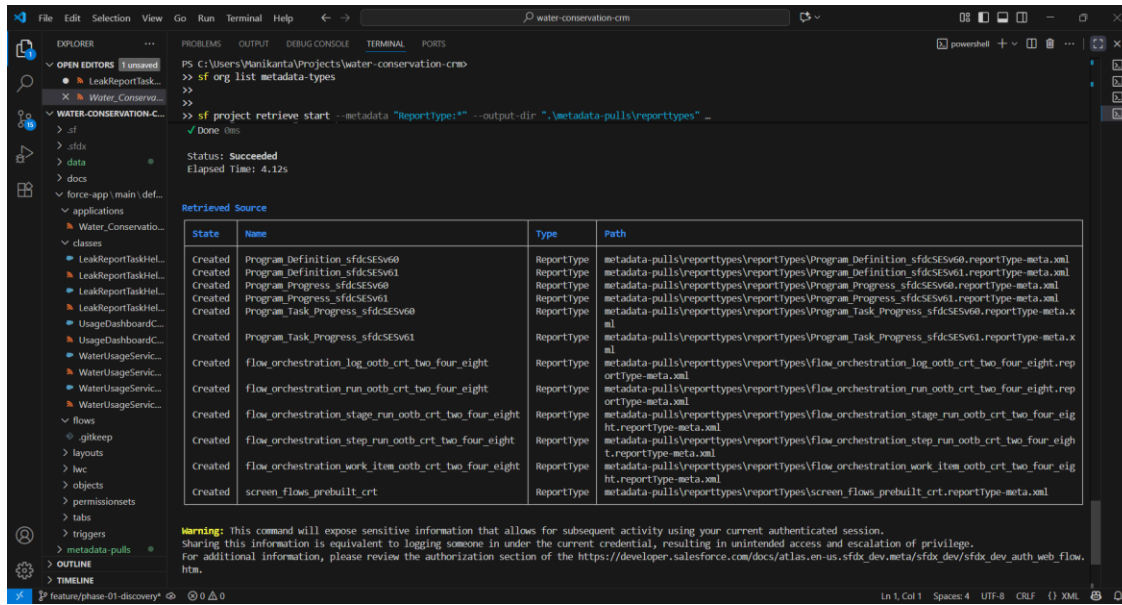


Fig 12: DEPLOYMENT

Phase 9: Reporting, Dashboards & Security Review

Objective

Deliver actionable analytics and complete a comprehensive security review prior to production launch.

Key Deliverables

- Report types and curated report library
- Executive dashboards (adoption, throughput, savings)
- Row-level security verified (sharing, filters)
- Security review artifacts: threat model, Health Check, vulnerabilities remediated

Activities

- Define KPIs aligned to program goals; map to fields and report types.
- Build dashboard pages for executives, program managers, and operations.
- Validate field-level and record-level access for reports.
- Run security assessments: session settings, CSP, clickjack, content sniffing.
- Static/dynamic scans (PMD, dependency checks, OWASP guidance for custom endpoints).

Roles

- Reporting Analyst, Admin, Security Architect, Product Owner.

Acceptance & Exit Criteria

- Dashboards approved by stakeholders; performance acceptable.
- No high/critical security findings outstanding.
- Data access in reports matches intended visibility.

Risks & Mitigations

- Misleading metrics → clear definitions and filters; data provenance notes.
- Overexposed data in reports → report folders, row-level filters, OWD review.

Checklist

- Report types curated
- Dashboard filters and sources validated
- Health Check $\geq$ target
- Vulnerabilities triaged and fixed

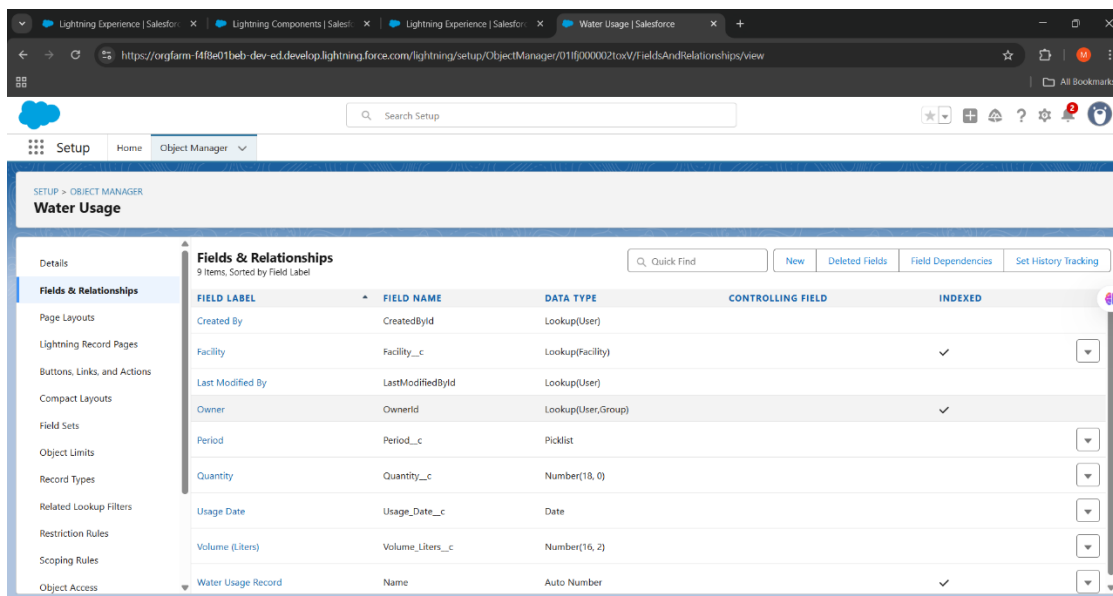


Fig 13: OBJECTS & RELATIONSHIPS

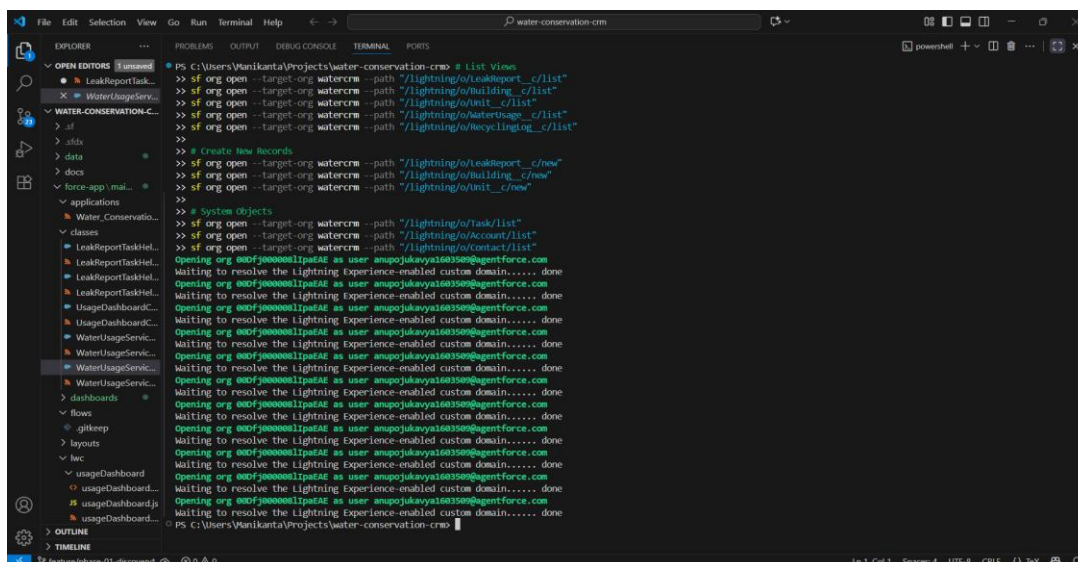


Fig 14: REPORTING

Phase 10: Quality Assurance Testing

Objective

To ensure the Water Resource Monitoring and Management system delivers accurate, secure, and reliable insights into water usage, leak detection, and conservation practices. The QA phase validates that the solution meets business objectives, complies with regulatory standards, and operates at scale with minimal defects.

Activities

1. Governance & Compliance Testing

- Validate data collection and aggregation across buildings/departments.
- Confirm architecture/design reviews for scalability.
- Ensure compliance with data retention, audit trails, and consent tracking.
- Verify encryption of sensitive water usage and building data.

2. Access & Security Testing

- Apply principle of least privilege (admins vs. facility managers vs. end-users).
- Test field-level security for water usage reports.
- Run penetration and vulnerability tests on APIs handling sensor/leak data.

3. Data Protection & Observability

- Validate event monitoring for leak detection alerts.
- Confirm error dashboards and login monitoring are functional.
- Ensure audit logs track changes to conservation/recycling reports.

4. Testing Strategy

Unit Tests:

- Apex code for water usage calculations.
- LWC Jest tests for dashboards.
- Flow test templates for conservation workflows.

Integration Tests:

- API tests for IoT sensors → Salesforce data ingestion.

- Event flow tests for leak detection alerts.

User Acceptance Testing (UAT):

- Scripts validating business scenarios like:
 - "Detect leak in Building A within 10 minutes."
 - "Generate recycling usage report per department."

Performance Testing:

- Validate response time with large water usage datasets.
- Measure query efficiency for historical trend analysis.

5. Automation & Pipelines

- CI: Static code analysis (PMD, ESLint), automated unit tests.
- CD: Deploy manifests for water monitoring objects, assign permission sets.
- Release: Semantic versioning with release notes highlighting conservation KPIs.

6. Back-Out & Recovery

- Rollback plans for failed releases (sensor integration, dashboard updates).
- Data backups before metadata updates.
- Change freeze windows during peak water usage monitoring (e.g., summer).
- Incident response with predefined escalation paths.

Roles

- QA Lead: Oversees testing strategy, ensures compliance.
- Developers: Write Apex/LWC unit tests, fix defects.
- Business Analysts: Define UAT scripts with conservation goals.
- End Users (Facilities Managers): Validate real-world leak detection and reporting.
- DevOps Engineer: Manage CI/CD pipelines, rollback, and deployments.
- Security Officer: Review encryption, audit trails, and access compliance.

Checklist

Governance

- Architecture and design reviews completed.
- Change control approvals documented.

Access

- Permissions aligned with least privilege.
- Field-level security reviewed for water usage objects.

Data Protection

- Sensitive water usage data encrypted.
- Event Monitoring enabled for leaks and anomalies.

Compliance

- Data retention policy validated.
- Consent tracking for stakeholder reports.
- Legal holds process defined.

Observability

- Login and audit logs enabled.
- Leak detection events visible in dashboards.
- Error logs without PII confirmed.

Quality, Testing & Release Management KPIs

- Adoption: Number of facility managers using dashboards weekly.
- Process Efficiency: Leak detection → resolution cycle time.
- Data Quality: Accuracy of water usage readings, duplicate record rate.
- Reliability: Deployment success rate, MTTR for leak detection service.
- Impact:
 - Reduction in water wastage (measured %).
 - Increase in recycling/reuse reporting accuracy.
 - Verified vs. estimated water savings.

This QA phase ensures the system not only works technically but also delivers measurable conservation outcomes that directly address the problem statement of water scarcity.

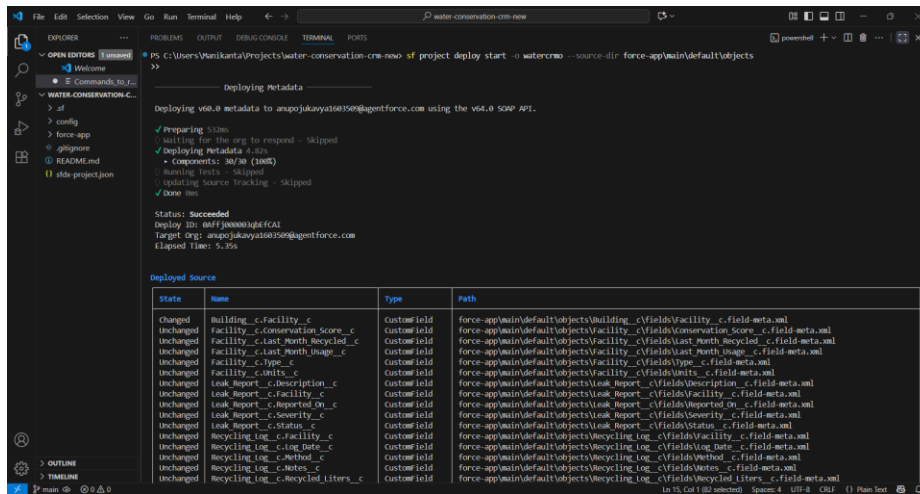


Fig 15: DEPLOYED SOURCE

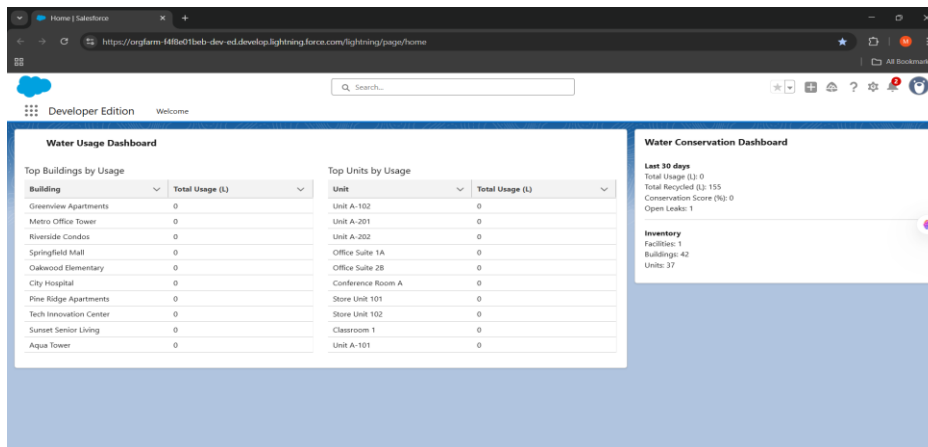


Fig 16: WATER USAGE DASHBOARD

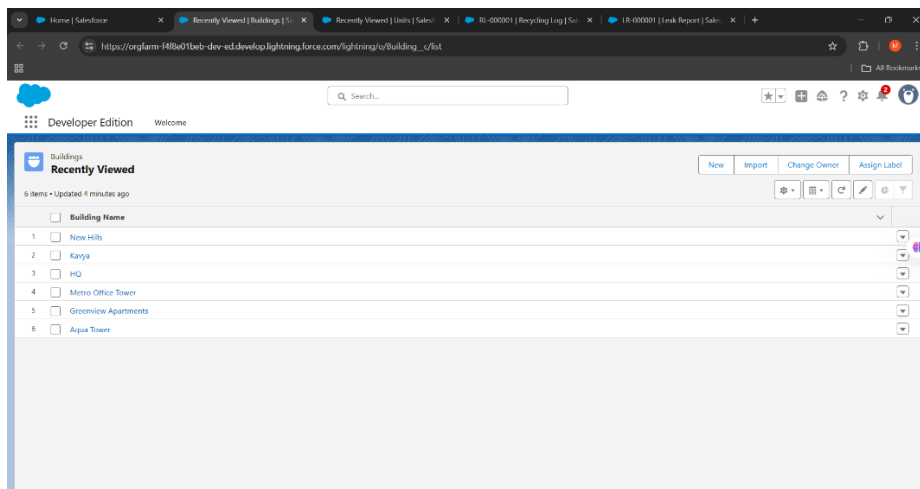


Fig 17: RECENTLY VIEW BUILDINGS

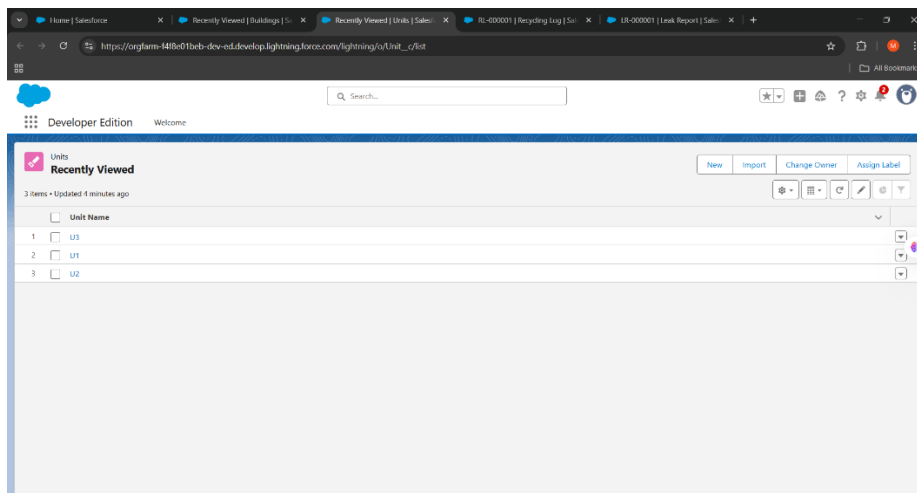


Fig 18: RECORDS

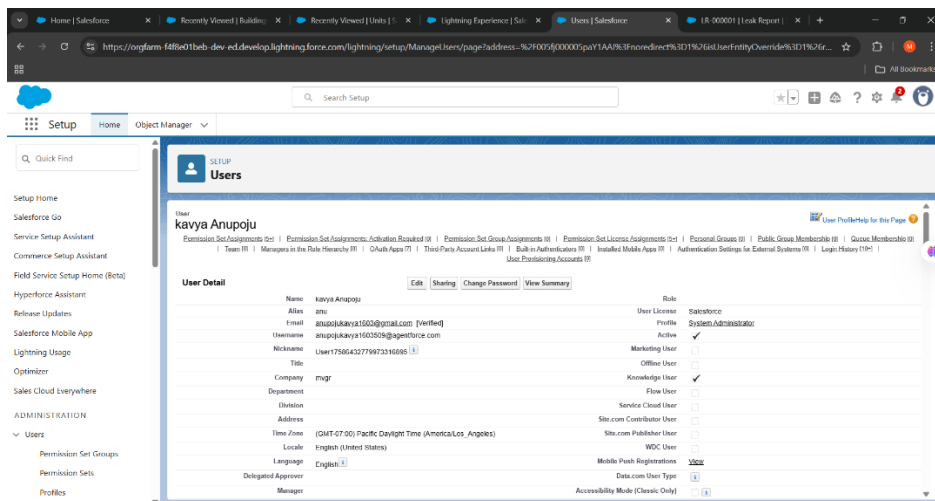


Fig 19: USER PAGE

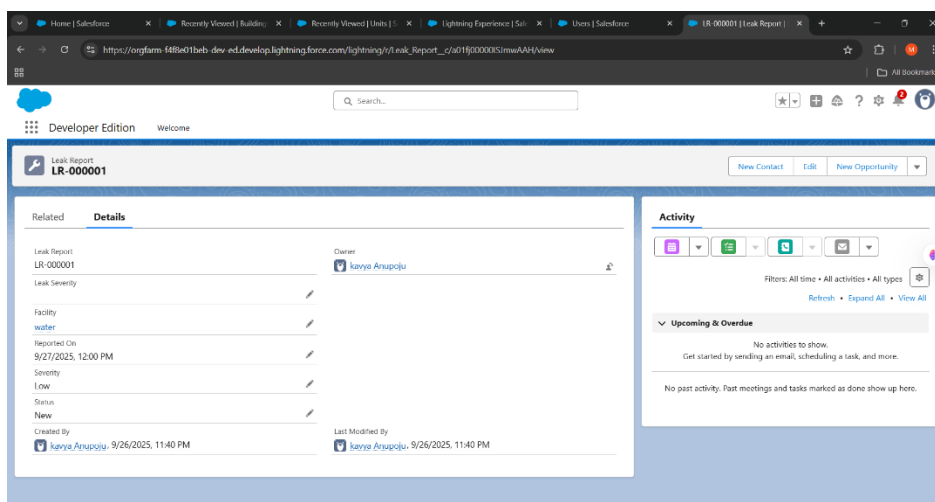


Fig 20: LEAKAGE REPORT

Conclusion

The Water Resource Monitoring and Management System successfully delivers a centralized, reliable, and secure platform for organizations to track and optimize water usage. By enabling real-time leak detection, organizations can minimize wastage and reduce operational costs. The dashboards provide clear visibility into water consumption, recycling, and reuse patterns, empowering facility managers and leadership to make data-driven decisions. With improved sustainability reporting and compliance readiness, the system enhances accountability and aligns with environmental conservation goals. Additionally, its scalable architecture ensures readiness for future integration with advanced IoT devices and analytics tools, making it a proactive and long-term solution to water scarcity challenges.

Future Enhancements

- **IoT Expansion:** Seamless integration with smart meters, advanced leak sensors, and AI-enabled valves for automated water shutoff.
- **AI/ML Analytics:** Predictive models to forecast consumption trends, detect anomalies, and suggest conservation strategies.
- **Mobile Application:** Real-time alerts, usage tracking, and reporting accessible on mobile devices for end-users and administrators.
- **Gamification & Awareness:** User engagement features like conservation scorecards and department-level challenges to promote water-saving habits.
- **Integration with ESG Reporting:** Automated alignment with global sustainability frameworks (e.g., GRI, CDP) for organizational reporting.
- **Cloud & Edge Computing:** Leveraging edge devices for faster on-site leak detection and cloud scalability for global organizations.
- **Multi-Utility Monitoring:** Extending the platform to cover electricity and energy usage alongside water, creating a holistic resource management system.